

REST API Fundamentals – Detailed Guide

REST API (Representational State Transfer Application Programming Interface) is one of the most widely used architectural styles for building modern web services. It enables communication between different systems over the HTTP protocol. In a typical web application, a frontend client such as a web browser or mobile app communicates with a backend server using REST APIs to fetch, create, update, or delete data. REST APIs are designed around the concept of resources. A resource can be anything such as a user, order, product, or payment. Each resource is identified by a URL and manipulated using standard HTTP methods.

1. How REST APIs Work

In modern applications, the client and server communicate through HTTP requests and responses.

Step-by-step flow:

1. A client (React app, mobile app, etc.) sends an HTTP request to a server.
2. The request contains an endpoint URL, HTTP method, headers, and optionally a request body.
3. The server processes the request, interacts with business logic and databases.
4. The server returns a response containing data and an HTTP status code.

Example request:

GET /users/101

Example JSON response:

```
{
  "id": 101,
  "name": "Faiz",
  "city": "Nagpur"
}
```

This allows the frontend application to display the information received from the server.

2. HTTP Methods Used in REST APIs

Method	Purpose	Example
GET	Retrieve data from the server	GET /users
POST	Create a new resource	POST /orders
PUT	Update an entire resource	PUT /users/1
PATCH	Update part of a resource	PATCH /users/email
DELETE	Remove a resource	DELETE /users/1

3. What is JSON?

JSON (JavaScript Object Notation) is the most common format used to exchange data between the client and server.

Advantages of JSON:

- Lightweight and compact
- Easy for humans to read
- Language independent
- Easily parsed by programming languages

Example JSON:

```
{  
  "name": "Faiz",  
  "age": 25,  
  "skills": ["Java", "Spring Boot", "React"]  
}
```

Frameworks like Spring Boot automatically convert JSON data to Java objects using libraries like Jackson.

4. Stateless Nature of REST

REST architecture is stateless. This means the server does not store any information about previous client requests.

Each request must contain all the information required to process it.

Example:

A user request must include authentication information such as a token in every request.

Authorization: Bearer

Benefits of stateless systems:

- Improved scalability
- Easier load balancing
- Simplified server design
- Better reliability in distributed systems

5. HTTP Response Status Codes

When a server processes a request, it returns an HTTP status code indicating the result.

Common categories:

1xx – Informational responses

2xx – Success

200 OK – Request successful

201 Created – Resource created successfully

204 No Content – Success but no response body

3xx – Redirection

301 Moved Permanently

302 Found

4xx – Client Errors

400 Bad Request – Invalid request

401 Unauthorized – Authentication required

403 Forbidden – Access denied

404 Not Found – Resource not found

5xx – Server Errors

500 Internal Server Error – Generic server error

502 Bad Gateway – Invalid response from upstream server

503 Service Unavailable – Server temporarily unavailable

Conclusion

REST APIs form the backbone of modern web and mobile applications. They enable scalable, flexible, and loosely coupled communication between systems. By using standardized HTTP methods, JSON data formats, stateless communication, and clear status codes, developers can build reliable services that power applications such as e-commerce platforms, social media apps, payment systems, and cloud services. Understanding REST APIs is essential for backend developers, especially when working with frameworks like Spring Boot, Node.js, or Django, where RESTful services are commonly used to build microservices and distributed systems.